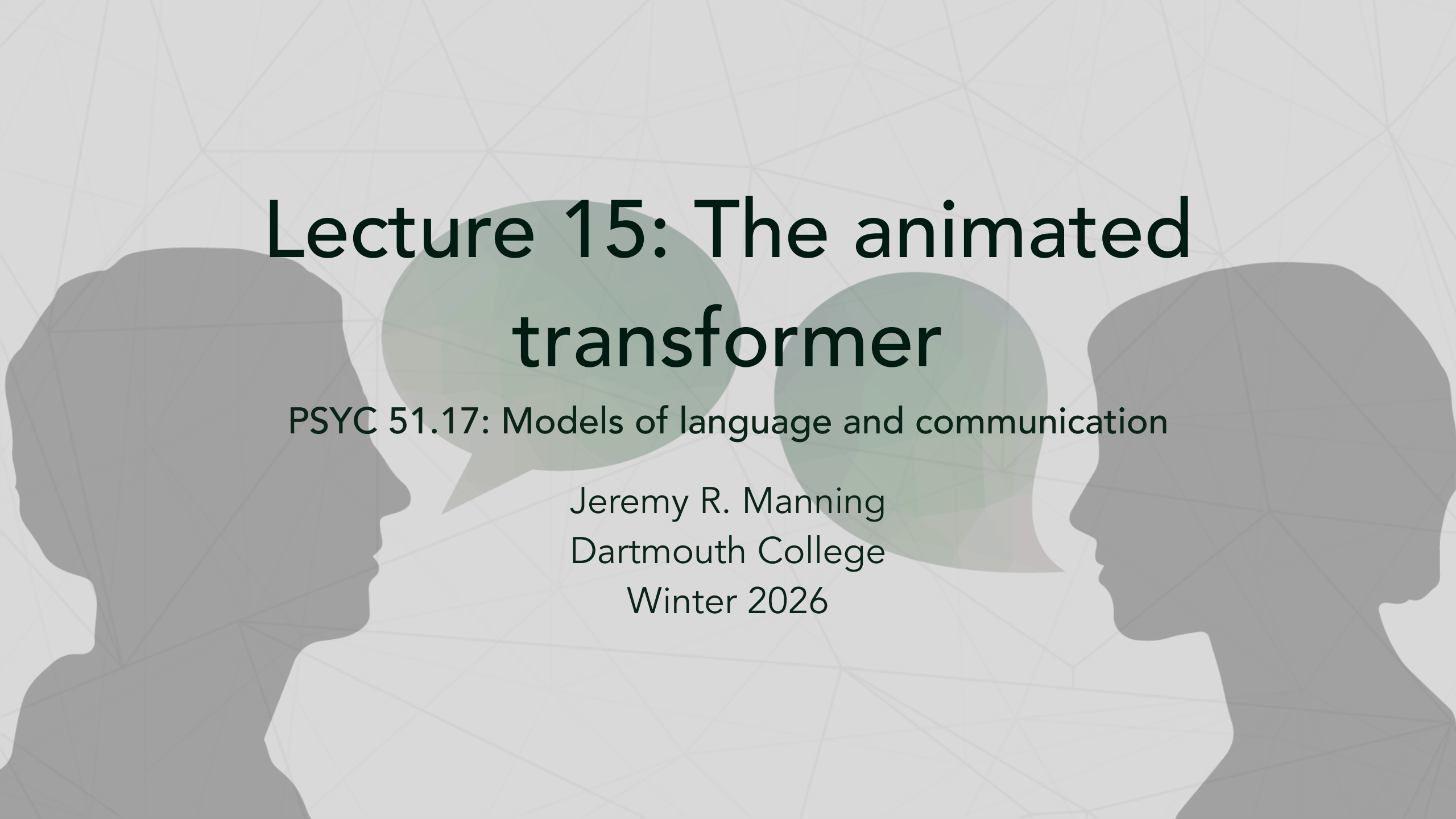
Lecture 15: The animated transformer

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026



Learning objectives

By the end of this lecture, you will

1. Understand the Transformer as a function that predicts the next word
2. Follow data from raw text through tokenization, embedding, and attention
3. Explain how queries, keys, and values enable self-attention
4. See how multiple attention heads capture different patterns
5. Understand how stacking blocks creates deep representations

What is a transformer?

The Transformer is a machine learning model for **sequence modeling**. Given a sequence of *things*, the model can predict what the next *thing* in the sequence might be.

The big picture

We can think of the Transformer as a function: $\text{Transformer}(X, \theta) \to Y$

- X is our input sequence
- θ represents the model parameters
- Y is the predicted next token

The transformer function

Tokenization

How tokenization works

The Transformer operates on sequences, so we first **tokenize** the input phrase. One approach is to treat each word as a token.

The model doesn't understand words directly—it identifies tokens using unique numbers from a vocabulary.

Our running example

"the robots will bring" → [3206, 2736, 3657, 400]

Each word maps to a unique ID in the vocabulary.

1. Embeddings: numbers speak louder than words

For each token, the Transformer maintains a vector called an **embedding**. An embedding aims to capture the semantic meaning of the token—similar tokens have similar embeddings.

> **Recall from Lecture 11**
>
> This is the same idea as Word2Vec, but the embeddings are learned jointly with the rest of the model.

Token embeddings

Token embeddings

Dimensions

Our transformer has embedding vectors of length **C = 768**. All embeddings can be packed together in a single $T \times C$ matrix, where $T = 4$ is the number of input tokens.

Position embeddings

Position embeddings

Why positions matter

In order to capture the significance of the **position** of a token within a sequence, the Transformer also maintains embeddings for each position.

Without position information, "cat sat" and "sat cat" would look identical to the model!

Combined embeddings

Token embedding matrix
$T \times C$

the	0.07	0.13	0.63	...	0.23
robots	0.81	0.51	0.44	...	0.98
will	0.62	0.29	0.80	...	0.34
bring	0.50	0.16	0.93	...	0.19

+

Position embedding matrix
$T \times C$

0	0.32	0.12	0.88	...	0.75
1	0.14	0.07	0.41	...	0.35
2	0.07	0.47	0.23	...	0.86
3	0.81	0.12	0.06	...	0.24

Combined embeddings

Token + position = input

The token and position embedding matrices ($T \times C$ each) are **added together** to obtain a position-dependent embedding for each token.

The token and position embeddings are all part of θ, meaning they are tuned during model training.

2. Queries, keys, and values

The Transformer computes three vectors for each token: **query**, **key**, and **value**. This is done by multiplying with learned weight matrices:

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

The weight matrices W_Q, W_K, W_V are all part of θ.

Query, key, value projections

$$
\begin{array}{r}
 \\
\text{the} \\
\text{robots} \\
\text{will} \\
\text{bring}
\end{array}
\overset{T \times C}{\begin{bmatrix}
0.39 & 0.25 & \dots & 0.98 \\
0.95 & 0.58 & \dots & 1.33 \\
0.69 & 0.77 & \dots & 1.20 \\
1.31 & 0.28 & \dots & 0.43
\end{bmatrix}}
\times
\overset{W_q\ (C \times C)}{\begin{bmatrix}
0.02 & 0.21 & \dots & 0.37 \\
0.03 & 0.02 & \dots & 0.05 \\
0.29 & 0.07 & \dots & 0.99 \\
0.38 & 0.37 & \dots & 0.28 \\
\vdots & \vdots & \vdots & \vdots
\end{bmatrix}}
=
$$

What are Q, K, V?

Search engine analogy

Imagine you have a database of images with text descriptions:

- **Query:** The user's search text
- **Key:** The text descriptions in your database
- **Value:** The actual images

Only those images (values) whose descriptions (keys) best match the search (query) are returned.

Self-attention intuition

Self-attention works similarly—tokens "query" other tokens to find which ones they should pay attention to.

3. Two heads are better than one

The Transformer splits the Q, K, V matrices into multiple **heads**. With C = 768 columns and 12 heads, each head operates on 64 dimensions.

Why multiple heads?

Different heads can specialize in different patterns—syntax, coreference, semantic roles, etc. The model decides what's useful!

Splitting into heads

$$Q\ (T \times C)$$

$$\begin{bmatrix} 0.07 & 0.04 & \dots & 0.64 \\ 0.07 & 0.07 & \dots & 0.36 \\ 0.08 & 0.06 & \dots & 0.44 \\ 0.04 & 0.04 & \dots & 0.34 \end{bmatrix}$$

4. Time to pay attention

Self-attention is the core idea behind the Transformer.
We compute an **attention scores** matrix:

$$A = \frac{Q \cdot K^T}{\sqrt{d_k}}$$

This matrix tells us how much attention each token should pay to every other token.

Computing attention scores

$$
\begin{array}{r}
 \\
\text{the} \\
\text{robots} \\
\text{will} \\
\text{bring}
\end{array}
\begin{array}{c}
Q_1\ (T \times H) \\
\begin{bmatrix}
0.07 & 0.04 & \dots & 0.64 \\
0.07 & 0.07 & \dots & 0.36 \\
0.08 & 0.06 & \dots & 0.44 \\
0.04 & 0.04 & \dots & 0.34
\end{bmatrix}
\end{array}
\times
\begin{array}{c}
K_1\ (T \times H) \\
\begin{bmatrix}
0.22 & 0.13 & \dots & 0.10 \\
0.22 & 0.03 & \dots & 0.10 \\
0.20 & 0.04 & \dots & 0.11 \\
0.19 & 0.06 & \dots & 0.24
\end{bmatrix}
\end{array}
\begin{array}{l}
 \\
\text{the} \\
\text{robots} \\
\text{will} \\
\text{bring}
\end{array}
$$

5. Applying attention

$$A_1\ (T \times T)$$

	the	robots	will	bring
the	0.18	0.22	0.23	0.37
robots	0.18	0.29	0.30	0.24
will	0.18	0.26	0.28	0.28
bring	0.17	0.30	0.31	0.22

Applying attention

Causal masking

The attention score for a token needs to be **masked** if it occurs later in the sequence.

"bring" can pay attention to "robots", but not vice-versa—a token shouldn't look at future tokens when predicting its next token.

Three steps

1. **Mask** future tokens (set upper triangle to $-\infty$)
2. **Softmax** each row (normalize to probabilities)
3. **Multiply by V** (weighted sum of value vectors)

Computing outputs

Weighted output

How outputs are computed

The output for "robots" is a weighted sum of value vectors:

$$Y(\text{robots}) = 0.47 \cdot V(\text{the}) + 0.53 \cdot V(\text{robots})$$

Each token's new representation is informed by relevant context!

6. Feed forward

Everything so far has been **linear operations**. This isn't enough to capture complex relationships!

Adding non-linearity

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

The hidden layer expands to $4C = 3072$ dimensions, then projects back to $C = 768$.

Feed-forward network

Feed-forward network

Why it matters

All the weight matrices in the FFN are part of θ. Research suggests this is where factual "knowledge" is stored.

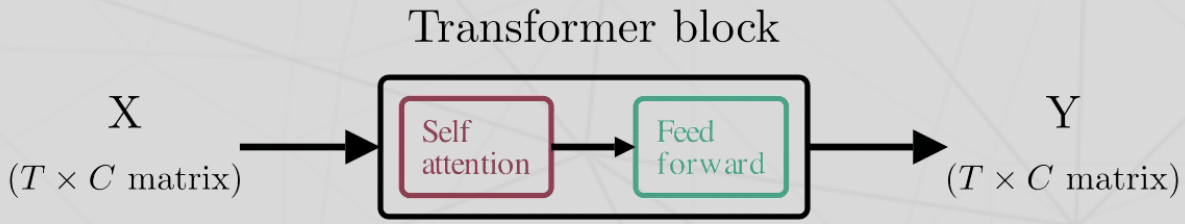
7. We need to go deeper

All the steps in sections 2–6 constitute a single **Transformer block**. Each block takes a $T \times C$ matrix as input and outputs a $T \times C$ matrix.

Model sizes

- GPT-2: 12–48 blocks
- GPT-3: 96 blocks
- GPT-4: Unknown (estimated 120+)

Stacking transformer blocks



8. Making a prediction

Making a prediction

The final step

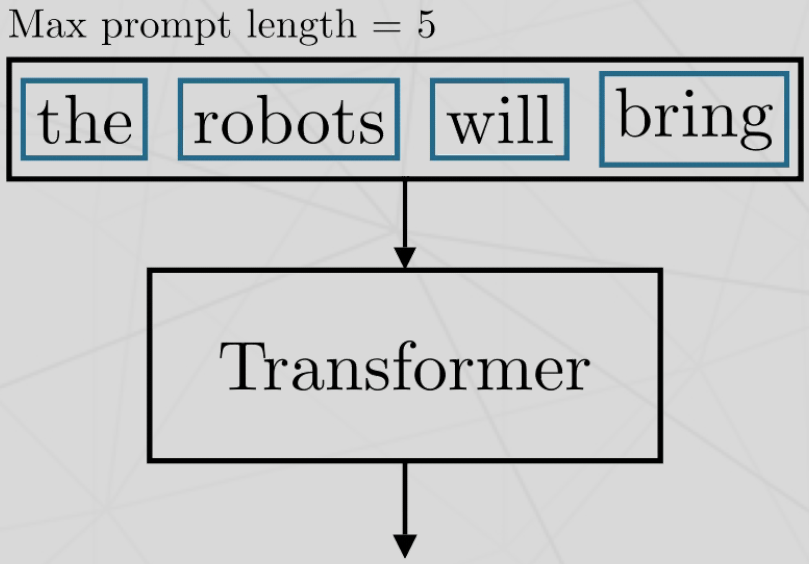
Take the last token's output vector and multiply by a $V \times C$ weight matrix, where V is the vocabulary size. Apply softmax to get a probability distribution over all words.

Our example

"prosperity" gets 92% probability, "destruction" only 5%.

So: "the robots will bring **prosperity**"

9. Text generator go brrr



Autoregressive generation

How text generation works

The first token produced is added to the prompt and fed back to produce the second token, which is then fed back to produce the third, and so on.

Context limit

Transformers have a maximum context length (N tokens). As generation continues, we may need to drop the oldest tokens.

And that's it!

The complete picture

1. **Tokenize** text to IDs
2. **Embed** tokens + positions
3. **Project** to Q, K, V
4. **Split** into attention heads
5. **Compute** attention scores
6. **Apply** masking and softmax
7. **Multiply** by V for output
8. **Feed forward** with non-linearity
9. **Stack** many blocks
10. **Predict** next token

What we left out

Details for another day

To focus on the most important aspects, we skipped:

- **Layer normalization** (stabilizes training)
- **Residual connections** (helps gradient flow)
- **Dropout** (regularization)
- **Training** (how θ is learned)

See nanoGPT for a complete implementation.

References

Further reading

Vaswani et al. (2017) "Attention Is All You Need" — The original transformer paper.

The Animated Transformer — Visual walkthrough that inspired this lecture.

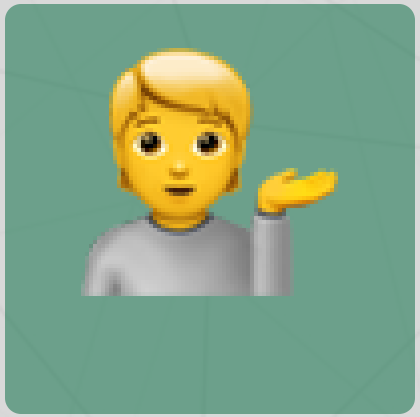
The Illustrated Transformer — Jay Alammar's detailed visual guide.

nanoGPT — Andrej Karpathy's minimal GPT implementation.

Questions?

Email

Discord



Office hours

Up next...

Lecture 16: Training transformers — loss functions, optimization, and scaling laws